



电子科技大学

University of Electronic Science and Technology of China

信息与软件工程学院

SCHOOL OF INFORMATION AND SOFTWARE ENGINEERING

Linux网络编程-线程池处理并发请求

Linux下HTTP Server的设计

- 训练目的
- 训练原理
- 训练内容



- 掌握并发编程基本理论：该实验旨在帮助学生深入理解并发编程的核心概念，包括并发和并行、线程与进程、同步与异步等，并能够理解这些概念在实际编程中的应用。
- 学习线程池的概念和应用：线程池是处理并发请求中常用的一种技术，能够有效地管理和控制线程的数量，提高系统的性能。通过这个实验，学生将学习如何在Linux环境下创建和使用线程池。

- 理解并实践客户端-服务器模型：该实验通过设计一个处理并发请求的服务器，使学生对客户端-服务器模型有更深入的理解。学生将了解到如何处理来自客户端的并发请求，以及如何分配和管理服务器的资源。

listen_fd -> accept() -> client_fd -> task queue -> worker -> handler -> close(client_fd)

监听套接字 -> 接收连接 -> 客户端套接字 -> 任务队列 -> 工作线程 -> 请求处理 -> 关闭连接

- ① listen_fd: 服务器启动后创建的监听套接字，绑定 IP 和端口后持续等待客户端连接。
- ② accept(): 主线程调用它接收一个新的客户端连接。
- ③ client_fd: accept() 成功后返回的新套接字。它只对应当前这个客户端，用于后续读请求、写响应。
- ④ task queue: 主线程不直接处理请求，而是把 client_fd 放入任务队列，等待 worker 领取。
- ⑤ worker: 线程池中的工作线程从任务队列取出一个 client_fd。队列为空时，worker 阻塞等待，不占用 CPU 忙等。
- ⑥ handler: worker 调用请求处理函数，例如解析 HTTP 请求、读取文件、生成响应并发送给客户端。
- ⑦ close(client_fd): 当前请求处理结束后，由 worker 关闭这个客户端连接，释放文件描述符资源。

其中 listen_fd 通常始终由主线程保留，用于继续接收新连接；client_fd 则在成功入队后交给 worker 处理并关闭。

- 并发：指两个或更多的任务能够在重叠的时间段内启动、运行和完成。在单核处理器中，虽然一次只能执行一个任务，但是操作系统可以通过任务切换使得这些任务看上去是在"同时"运行，这就是并发。
- 并行：指两个或更多的任务同时执行。在多核处理器中，任务可以在不同的处理器核心上同时执行，这就是并行。

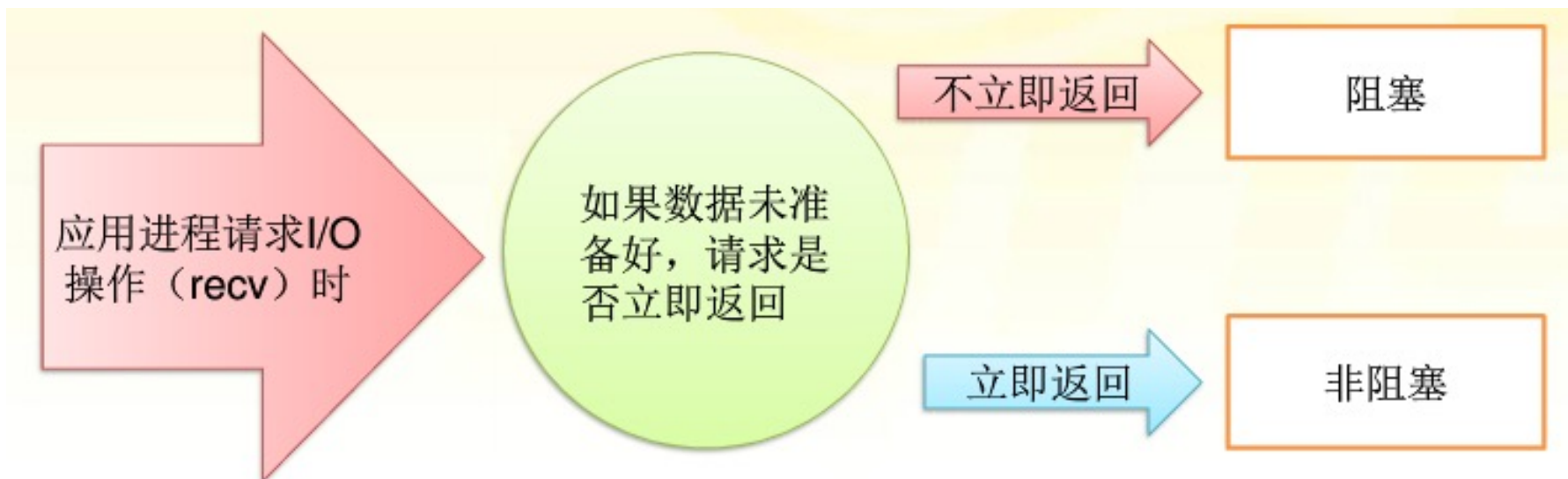
- 进程是操作系统**资源分配**的基本单位，
- 线程是操作系统**调度（CPU分配）**的基本单位。
- 每个进程拥有自己独立的内存空间和系统资源，而线程则是在同一进程内共享资源，每个线程拥有自己的运行栈和状态。

- ◆同步是指一个操作（或称为任务）必须等待另一个操作完成后才能开始
- ◆异步是指一个操作可以在另一个操作开始后，不必等待其完成就能开始。

这两个概念常常用在I/O操作，例如文件读写，网络数据传输等。

是任务之间的关系

阻塞和非阻塞通常描述的是程序等待调用结果（如I/O操作）的行为。



- ◆阻塞调用是指调用结果返回之前，当前线程会被挂起。
- ◆非阻塞调用指在不能立即得到结果之前，该调用不会阻塞当前线程。

- ◆在并发环境下，多个线程经常需要访问和修改共享数据，这可能导致数据不一致或者其它问题（如死锁）。
- ◆为了保护共享数据的一致性，通常需要使用互斥和同步原语，如互斥量（mutex）、信号量（semaphore）、条件变量等。

互斥量在设计时，只有两种状态：锁定和解锁。

- ◆当线程想要访问共享资源时，它首先会尝试锁定与该资源相关联的互斥量。
- ◆如果互斥量已被其他线程锁定，该线程将阻塞，直到互斥量被解锁。
- ◆如果互斥量未被锁定，那么该线程将锁定互斥量，并继续访问资源。
- ◆当访问完成后，线程将解锁互斥量，以便其他线程可以访问该资源。

◆信号量 (Semaphore)：是一个用来控制多个线程对有限资源访问的计数器。与互斥量类似，但它允许多个线程同时访问同一资源。信号量的值代表可用资源的数量。

◆条件变量（Condition Variables）：

用于让线程在某些条件不满足时等待，直到其他线程通知条件已经满足，使得线程之间可以按照某种顺序执行。

例如，一个线程在缓冲区满时可能需要等待，直到另一个线程消耗了缓冲区中的一部分数据。

◆死锁是指两个或多个任务互相等待对方持有的资源，导致都无法继续执行。

◆饥饿是指一个或多个任务由于某种原因无法获得所需资源，导致无法继续执行。

这两个问题都是并发编程中需要避免的。

避免死锁：

- ◆锁定顺序：设定并且坚持一个确定的锁定顺序。如果所有线程总是以相同的顺序获取锁，那么就不可能产生循环等待，从而避免了死锁。
- ◆锁定超时：设定一个锁的获取超时时间。如果线程在预定时间内无法获取所有必需的锁，那么它就释放所有已经获取的锁，然后等待一段随机时间后重试。

避免死锁：

- ◆ 死锁检测：通过检测系统的资源分配状态，如果检测到环路等待现象，就可以进行相应的处理，如撤销部分操作，或者重新调度。
- ◆ 避免占有并等待：线程一次性申请所有必需的资源，只有得到所有资源后才开始执行，否则就不申请。

避免饥饿：

- ◆ 公平锁：使得等待时间最长的线程优先获取资源。在很多编程语言和操作系统中，可以设定锁为公平锁，使得线程按照它们请求锁的顺序来获取锁。
- ◆ 优先级调度：根据线程的优先级来分配资源，优先级高的线程更有可能获取到资源。但这需要注意，过度依赖优先级调度可能导致低优先级的线程长期得不到执行（优先级反转或优先级饥饿）。

避免饥饿：

- ◆ 资源预留：对资源的需求大的线程，可以预留一部分资源，避免因竞争少量资源而导致的饥饿。
- ◆ 防止线程过度占用资源：通过适当的调度策略，例如轮转法（Round Robin）或者抢占式调度，保证每个线程都有运行的机会。

多线程面临的两个问题：

- ◆ 如果创建很多生命期很短的线程，就会消耗很大的系统资源，甚至给系统带来很大的压力。
- ◆ 系统能承受的线程数量是有限的，创建大量的线程会大大降低性能甚至崩溃。

线程池是一种多线程处理形式，处理过程：

- ① 通过预先创建一定数量的线程并放入线程池中，
- ② 有任务需要执行时，从线程池中取出线程处理任务，
- ③ 任务执行完毕后，线程再放回线程池中等待下一次任务

线程池的一个重要优点是可以控制系统中并行执行的线程的数量。也就是可以限制需要并行处理的线程的数量。

使用的场景：

- ① 需要大量的线程来完成任务，且完成任务的时间比较短；
- ② 对性能要求苛刻的应用；
- ③ 接受突发性的海量请求，但不至于使服务器因此产生大量线程的应用。

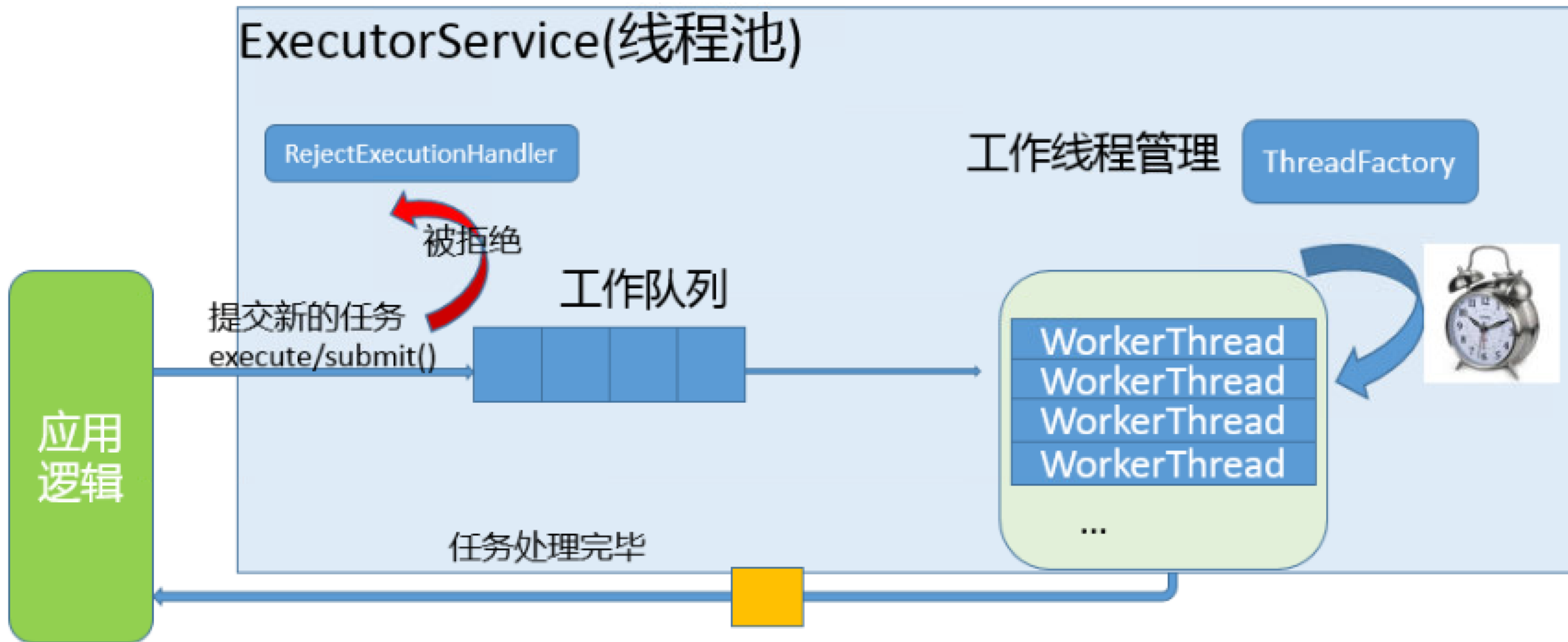
“池”的概念是一种非常常见的空间换时间的概念，除了有线程池之外还有进程池、内存池等等。核心思想都是先申请一批资源出来，然后就随用随拿，不用释放。

线程池的工作流程如下：

- ① 在线程池启动后，线程池中的线程就开始等待任务。
- ② 当调用者提交任务给线程池时，线程池中的线程就会被唤醒执行任务。如果所有线程都在执行任务，那么新的任务就会等待（通常在一个队列中）。
- ③ 一旦线程完成了它的任务，它将再次等待线程池中的新任务。

线程池的优点包括：

- ◆ 重用线程，避免了因为线程的创建和销毁所带来的性能开销。
- ◆ 可以控制系统中的并行线程数量，避免大量线程之间的切换导致的性能开销。
- ◆ 可以根据系统的承受能力，调整线程的并行数量。
- ◆ 提高了系统的响应速度，任务不必等待线程创建就能立即执行。



线程池的组成主要分为 3 个部分，这三部分配合工作就可以得到一个完整的线程池：

- 1、**任务队列**，存储需要处理的**任务**，由工作的线程来处理这些任务
 - ✓ 通过线程池提供的 API 函数，将一个待处理的任务添加到任务队列，或者从任务队列中删除
 - ✓ 已处理的任务会被从任务队列中删除
 - ✓ 线程池的使用者，也就是调用线程池函数往任务队列中添加任务的线程就是生产者线程

2、工作的线程（任务队列任务的消费者），N个

- ✓ 线程池中维护了一定数量的工作线程，他们的作用是是不停的读任务队列，从里边取出任务并处理
- ✓ 工作的线程相当于是任务队列的消费者角色，
- ✓ 如果任务队列为空，工作的线程将会被阻塞 (使用条件变量 / 信号量阻塞)
- ✓ 如果阻塞之后有了新的任务，由生产者将阻塞解除，工作线程开始工作

3. 管理者线程（不处理任务队列中的任务），1个

- ✓ 它的任务是周期性的对任务队列中的任务数量以及处于忙状态的工作线程个数进行检测
- ✓ 当任务过多的时候，可以适当的创建一些新的工作线程
- ✓ 当任务过少的时候，可以适当的销毁一些工作的线程

Socket编程的基本流程：

- 创建Socket：首先，需要调用一个函数（在C语言中是socket()）来创建一个新的Socket。这个函数会返回一个指向新创建的Socket的句柄。
- 绑定Socket：创建Socket后，需要将它绑定到一个特定的网络地址和端口上。这是通过bind()函数完成的。
- 监听连接：在服务器端，需要调用listen()函数来开始监听来自客户端的连接请求。
- 接受连接：服务器在收到客户端的连接请求后，会通过accept()函数来接受这个连接。
- 发送和接收数据：一旦连接建立，服务器和客户端就可以通过send()和recv()函数（或write()和read()函数）来发送和接收数据。
- 关闭连接：当数据传输完成后，双方都需要关闭连接。这是通过close()函数（或shutdown()函数）完成的。

V0.7：每个客户端连接 fork 一个子进程

问题：请求多时，进程数量失控，创建和销毁成本高

V0.8：固定数量 worker 线程

主线程只 accept

worker 线程从任务队列取 client_fd

- 首先，需要一个线程池结构，以便跟踪线程，工作队列等：

```
1  typedef struct {  
2      pthread_mutex_t mutex;    // 用于同步的互斥锁  
3      pthread_cond_t cond;      // 条件变量，用于线程之间的同步  
4      pthread_t *threads;       // 线程数组  
5      work_t *work_queue;       // 工作队列（链表形式）  
6      int count;                // 工作队列中的工作数量  
7      int thread_count;         // 线程数量  
8      int shutdown;             // 标记是否关闭线程池  
9      int started;              // 已经启动的线程数量  
10 } threadpool_t;
```

■ 其中work_t的最小结构可参考：

```
typedef struct {  
    int client_fds[64];  
    int head;  
    int tail;  
    int count;  
    int shutdown;  
    pthread_mutex_t mutex;  
    pthread_cond_t not_empty;  
} work_t;
```

■ 然后，创建线程池和线程：

```
1  threadpool_t *pool = malloc(sizeof(threadpool_t)); // 分配线程池
2  pool->thread_count = num_threads;
3  pool->shutdown = 0;
4  pool->started = 0;
5
6  // 创建线程数组
7  pool->threads = malloc(sizeof(pthread_t) * num_threads);
8
9  // 初始化互斥锁和条件变量
10 pthread_mutex_init(&(pool->mutex), NULL);
11 pthread_cond_init(&(pool->cond), NULL);
12
13 // 启动线程
14 for(i = 0; i < num_threads; i++) {
15     pthread_create(&(pool->threads[i]), NULL, threadpool_thread, (void*)pool);
16     pool->thread_count++;
17     pool->started++;
18 }
```


■ 线程函数 (threadpool_thread) 处理工作队列中的工作：

```
1  void *threadpool_thread(void *threadpool) {
2      threadpool_t *pool = (threadpool_t *)threadpool;
3      work_t *work;
4
5      while(1) {
6          pthread_mutex_lock(&(pool->mutex));
7
8          // 等待直到有工作可做或线程池被销毁
9          while((pool->count == 0) && (!pool->shutdown)) {
10             pthread_cond_wait(&(pool->cond), &(pool->mutex));
11         }
12
13         if(pool->shutdown) {
14             break;
15         }
16     }
```

■ 线程函数 (threadpool_thread) 处理工作队列中的工作 (续)：

```
17 // 获取工作队列中的工作
18 work = pool->work_queue;
19 pool->work_queue = pool->work_queue->next;
20 pool->count--;
21
22 pthread_mutex_unlock(&(pool->mutex));
23
24 // 执行工作函数
25 (*(work->func))(work->arg);
26
27 // 释放工作结构
28 free(work);
29 }
30
31 pool->started--;
32
33 pthread_mutex_unlock(&(pool->mutex));
34 pthread_exit(NULL);
35 return(NULL);
36 }
```

结合服务器使用线程池

- ① 创建一个固定大小的线程池。
- ② 服务器监听新的客户端连接。当一个新的连接请求到来时，服务器接受这个连接，创建一个包含了必要信息（例如，客户端的套接字描述符）的任务，并将其添加到任务队列中。
- ③ 线程池中的线程在没有任务时会被阻塞。当任务队列中有新的任务时，一个线程会取出这个任务并执行它。在这种情况下，执行任务可能涉及到读取来自客户端的数据，处理这些数据，并将结果发送回客户端。
- ④ 当任务完成时，线程会返回到线程池并等待下一个任务。如果客户端断开连接，服务器将关闭相应的套接字描述符。

- 首先，定义线程池的大小和任务队列的结构体，用于同步的互斥锁和条件变量：

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <pthread.h>
4  #include <unistd.h>
5  #include <sys/socket.h>
6  #include <netinet/in.h>
7
8  #define NUM_THREADS 5
9  #define MAX_QUEUE_SIZE 10
10
11  struct job {
12      int client_socket;
13      struct job* next;
14  };
15
16  struct job* jobs = NULL;
17  pthread_mutex_t jobs_mutex = PTHREAD_MUTEX_INITIALIZER;
18  pthread_cond_t jobs_cond = PTHREAD_COND_INITIALIZER;
```

- 接下来创建一个worker线程函数，这个函数会一直循环并等待任务队列中出现新的任务：

```
1 void* worker_thread(void* arg) {
2     while (1) {
3         struct job* job;
4         pthread_mutex_lock(&jobs_mutex);
5         while (jobs == NULL) {
6             pthread_cond_wait(&jobs_cond, &jobs_mutex);
7         }
8         job = jobs;
9         jobs = job->next;
10        pthread_mutex_unlock(&jobs_mutex);
11        // 在这里处理客户端请求
12        char buffer[256];
13        int n = read(job->client_socket, buffer, 255);
14        if (n < 0) {
15            perror("ERROR reading from socket");
16        }
17        printf("Here is the message: %s\n", buffer);
18        // 回复客户端
19        n = write(job->client_socket, "I got your message", 18);
20        if (n < 0) {
21            perror("ERROR writing to socket");
22        }
23        close(job->client_socket);
24        free(job);
25    }
26    return NULL;
27 }
```

- 然后，在主函数中创建一个服务器socket，并监听新的客户端连接。每当一个新的客户端连接上来，就创建一个新的任务并添加到任务队列中：

```
int main(){  
    pthread_t threads [NUM_THREADS];  
    需求1：根据NUM_THREADS的数量，创建处理worker_thread线程，并放在threads  
    数组中。  
    需求2：根据socket编程框架，让本服务器监听在8080端口上。  
    While(1){  
        需求3：接收新客户端链接  
        需求4：malloc一个job节点，并将客户端链接赋值给job的client_socket  
        pthread_mutex_lock(&jobs_mutex);  
        需求5：将本节点插入到Jobs链表中。  
        Pthread_cond_signal(&jobs_cond);  
        Pthread_mutex_unlock(&jobs_mutex);  
    }  
    .....}
```

实验概述：

设计一个基于**TCP**协议的文本聊天程序，其中服务器端通过线程池来处理并发客户端的连接和消息通信请求。

实验要求：

1. 实现一个基于**TCP**协议的文本聊天程序，在服务器端使用线程池来处理客户端的并发请求。
2. 考虑聊天程序的实时通信性和稳定性，确保消息的及时传递和用户列表的正确维护。
3. 能够正确处理异常情况，比如连接断开或客户端退出，提供友好的提示和处理方式。
4. 可以考虑对客户端消息进行简单的处理，比如支持特殊指令、表情符号等功能。

实验具体内容：

1、服务器端设计：

- ① 服务器端启动时创建一个固定大小的线程池，用于处理客户端连接请求和消息通信。
- ② 每个线程从线程池中取出任务时，负责处理一个客户端连接，并实现与该客户端的文字通信。
- ③ 当有新的客户端连接请求时，服务器将该请求交由线程池中的一个空闲线程处理。
- ④ 服务器端需要保持所有在线客户端的连接，并能够向所有客户端广播收到的消息。
- ⑤ 线程在完成任务后返回到线程池中等待下一个任务。

2、客户端设计：

- ① 客户端启动时向服务器发起连接请求，建立**TCP**连接。
- ② 客户端可以发送文本消息给服务器，并接收服务器转发的其他客户端的消息并显示在界面上。
- ③ 客户端可以通过输入指令（如"**exit**"）来结束与服务器的通信并退出程序。

Webserver V0.8版

线程池版TCP Web Server



mini_webserver

工程基础模块 Day1 && Day4

- 构建子模块 D1
Makefile
编译、清理、调试版本
- 配置子模块 D1
Config.c
端口、根目录、线程数
- 日志子模块 D1,D4
log.c
访问日志、错误日志

用户数据模块 Day2 && Day3

- 用户结构子模块 D2-D3
user.c
链表、二叉树、增删改查
- CSV存储子模块 D2
csv_store.c
用户数据加载和保存
- 索引查找子模块 D3
user_index.c
按用户名快速查找

网络服务模块 Day6 && Day7

- TCP监听子模块 D6
server.c
socket, bind, listen
- 连接管理子模块 D7
client.c
连接与状态、关闭释放

并发处理模块 Day4-5 && Day8-9

- 进程模型子模块 D4
process.c
父子进程协同
- 线程同步子模块 D5
thread.c
锁、条件变量、共享资源保护
- 线程池子模块 D8
thread_pool.c
任务队列、工作线程
- 事件驱动子模块 D9
event_loop.c
select / epoll

HTTP业务模块 Day10-Day14

- 请求解析子模块 D10
http_request.c
方法、URL、Header、Body
- 静态资源子模块 D11
static_file.c
HTML、CSS、图片读取
- GET/POST处理子模块 D12
handler.c
表单提交、参数读取
- 路由配置子模块 D13
router.c
URL匹配、处理函数绑定
- 认证安全子模块 D14
auth.c
登录校验、权限控制、路径检查
- 响应生成子模块 D10-D14
http_response.c
状态码、响应头、响应体

测试模块 Day1-15

- 每日增加功能的相关测试
- 如: test_day01.sh
- 如: bench.c 并发压力测试/响应时间等

今日训练内容即为webserver要求内容:

增加线程池结构, 让服务器用固定数量 worker 处理客户端连接, 避免为每个连接临时创建进程或线程。

新增文件包括:

- `include/thread_pool.h`
- `src/thread_pool.c`
- `include/tcp_pool_server.h`
- `src/tcp_pool_server.c`
- `tests/test_day08.sh`

修改文件包括

- `src/main.c`
- `src/request_handler.c`
- `Makefile`
- `README.md`
- `docs/debug-log.txt` (可选)

V0.8版 mini_webserver

因此V0.7的多进程并发服务器要求：父进程持续接收客户端连接，每个客户端交给一个子进程处理。父进程要处理 SIGCHLD，避免僵尸进程；同时要处理 accept 被信号打断的问题。



V0.8版 mini_webserver-功能要求

- 主线程读取 conf/server.conf, 监听 127.0.0.1:8080
- 启动固定大小线程池
- 主线程循环accept
- 每个client_fd作为任务放入队列
- worker线程从队列取任务
- worker调用已有HTTP请求处理函数
- worker负责关闭client_fd
- 队列为空时worker阻塞等待
- 达到max_clients
- 后停止接收新连接关闭线程池, 唤醒 worker, 等待所有线程退出
- 日志记录 worker 线程编号、请求路径、响应状态

